

SLIIT · SE4040 ENTERPRISE APPLICATION DEVELOPMENT

VoltShare

Smart Solar Microgrid Trading System

Team Setup & Deployment Guide

How to go from the zip file you were sent to a working system on your own laptop — API, web application and Android app, all talking to each other.

Read Part 3 before you touch anything.

It explains why the addresses are what they are, and will save you an evening.

CONTENTS

What is in this guide

Work through the parts in order. Parts 1 and 2 install software and only need doing once. Part 3 is the explanation — it is the part that makes the rest make sense. Parts 4 to 9 are the actual setup, and take about twenty minutes once the tools are installed.

Part	What it covers	Once, or every time?
1	Software to install	Once
2	Turning on IIS in Windows	Once
3	How the three applications actually connect — read this	Once (understanding)
4	Unzip the project and find your IP address	Every time your IP changes
5	The three files you must edit for your machine	Every time your IP changes
6	Deploy the Web API to IIS	After any backend change
7	Deploy the web application to IIS	After any web change
8	The firewall trap — nearly everyone hits this	Once per network
9	Build and run the Android app	After any mobile change
10	Check it all works	Every time
11	Troubleshooting	As needed
12	Two things only the repository owner can do	Once, for the team

You need Windows

IIS is a Windows-only web server, and the assignment requires the API to be hosted on it. This guide assumes Windows 10 or 11. On a Mac you can still write code and run the Android app, but you cannot host the API on IIS.

PART 1

Install the software

Install these in the order given. The versions in the right-hand column are what the project was built and tested with — a newer patch release is fine, an older major version is not.

#	Software	Why it is needed	Version used
1	Git for Windows git-scm.com/download/win	Version control, and it gives you Git Bash.	2.54.0
2	.NET SDK 10 dotnet.microsoft.com/download/dotnet/10.0	Compiles the C# Web API. Pick <i>SDK</i> , not <i>Runtime</i> .	10.0.400
3	ASP.NET Core 10 Hosting Bundle same page, under "Run server apps"	Teaches IIS how to run a .NET application. Separate download from the SDK . Without it every request returns HTTP 500.19.	10.0.x
4	Node.js LTS nodejs.org	Builds the React web application. npm comes with it.	Node 25.8.1 npm 11.11.0
5	Android Studio developer.android.com/studio	Builds and runs the Android app. Bundles its own JDK and the Android SDK.	Ladybug or newer
6	MongoDB Compass (optional) mongodb.com/products/compass	A GUI for looking inside the database. Handy for screenshots in the report.	Any

Step 3 is the one people miss

The .NET SDK and the ASP.NET Core Hosting Bundle are two different downloads from the same page. Installing the SDK alone lets you build the API but not host it on IIS. After installing the Hosting Bundle, open an elevated PowerShell and run `iisreset` so IIS picks it up.

Check the installs worked

Open a new PowerShell window and run each of these. If a command is not recognised, that tool is not installed or is not on your PATH.

```
dotnet --version    -> 10.0.400
node --version      -> v25.8.1
npm --version       -> 11.11.0
git --version       -> git version 2.54.0.windows.1
```

And confirm the Hosting Bundle landed, which is the one that has no obvious command:

```
Test-Path "C:\Program Files\IIS\Asp.Net Core Module\V2\aspnetcorev2.dll"

True    <- good
False   <- go back and install the Hosting Bundle
```

PART 2

Turn on IIS

IIS ships with Windows but is switched off by default. It is a Windows feature, not a download.

- 1 Press **Start** and type **Turn Windows features on or off**. Open it.
- 2 Tick **Internet Information Services**. Ticking the parent box gives you the defaults, which is nearly enough.
- 3 Expand **Internet Information Services** → **World Wide Web Services** → **Application Development Features** and additionally tick **ISAPI Extensions** and **ISAPI Filters**.
- 4 Expand **Web Management Tools** and tick **IIS Management Console**. This is the graphical tool you will use to look at your sites.
- 5 Click **OK** and let Windows install. Restart if it asks.

Confirm it worked by opening a browser and going to:

```
http://localhost
```

You should see a blue **IIS Windows** welcome page. That is the default site sitting on port 80. Leave it alone — our two sites use their own ports and do not touch it.

Install the Hosting Bundle after IIS, not before

If you installed the ASP.NET Core Hosting Bundle before switching IIS on, the installer had no IIS to register itself with. Repair it from **Settings** → **Apps**, or simply run the installer again, then `iisreset`.

What IIS actually is

IIS stands for Internet Information Services. It is Microsoft's web server: a Windows program that listens on a port and hands out web pages and API responses. It is the same kind of thing as Apache or Nginx.

It is **not** a cloud service. When you deploy to IIS you are deploying to **your own laptop**. Your laptop becomes the server. This has one consequence that surprises everybody, and Part 3 explains it.

PART 3 · READ THIS ONE

How the three applications connect

This is the part that saves you time later. Five minutes here prevents a long evening of “it works on my laptop but not on my phone”.

Your laptop is the server

Services like Vercel or Netlify give you an address such as `myapp.vercel.app` because *their* computer, in *their* data centre, runs a web server 24 hours a day and they own a domain name pointing at it.

We are running the web server on our own laptops. So:

Your laptop must be switched on

Close the lid, sleep the machine, or leave the Wi-Fi network, and the application goes down for everyone. That is not a bug in the project — it is what self-hosting means. It is also exactly what the assignment asked for, because the brief requires the Web API to be hosted on IIS.

Three rings of reach

Every address answers from a different distance. This is the single most useful picture in this guide.

```
+-- the internet -----+
| needs a public IP or a cloud host |
|                                     |
| +-- your Wi-Fi (192.168.1.x) -----+
| | your phone, your teammate's laptop | |
| |                                     | |
| | +-- your laptop (localhost) -----+
| | | IIS :80    the blue default | | |
| | | IIS :8080  the Web API      | | |
| | | IIS :8081  the web application | | |
| | | vite :5173 dev server only  | | |
| | +-----+
| +-----+
+-----+
```

Address	Who can reach it	Use it for
http://localhost:8081	Only your own laptop. On a phone, <i>localhost</i> means the phone itself.	Testing on the machine you are sitting at.
http://192.168.1.190:8081	Anything on the same Wi-Fi — your phone, a teammate's laptop.	Demonstrations, and testing from a phone.
A public URL	Anyone on the internet.	You do not have one, and do not need one.

Do we need a public URL for this assignment?

No. Re-read the brief: it asks for a C# Web API hosted on IIS. Hosting on your own network, plus the demonstration video, is the expected deliverable. Marks come from the business logic living in the service, not from your DNS setup. Do not waste time on cloud hosting.

What sits on which port

After you finish Parts 6 and 7 your machine will look like this. The port numbers were chosen so that nothing collides with the IIS default site.

Port	What is there	Deployed by
:80	The blue IIS Windows default page. Untouched — we deliberately did not take port 80.	Windows
:8080	The Web API. Health check at <code>/api/v1/health</code> , Swagger at <code>/swagger</code> .	<code>deploy-api-iis.ps1</code>
:8081	The web application. This is the URL you give people.	<code>deploy-web-iis.ps1</code>
:5173	Vite's development server. A coding tool for you only — not part of the deployment.	<code>npm run dev</code>

Nobody clones the repository to use the web app

This confuses people, so it is worth being explicit. There are two completely different ways to run a React application.

Development mode — `npm run dev`, port 5173

A tool for you while you are writing code. It watches files and reloads the browser as you type. It needs Node, the repository and `node_modules`. **Users never touch this.**

Production mode — `npm run build`

Compiles the whole React application down to three plain files:

```
dist/
  index.html
  assets/index-<hash>.js    about 298 kB - all your React, bundled
  assets/index-<hash>.css   about 38 kB
```

No Node, no repository, no `npm install`. Any web server can hand those files to a browser, and the application then runs *inside the browser*. The deploy script does the build and the copy for you.

So the whole user journey is: **open a URL**. That is all.

Where each application gets the API address

Two of the three have the address **compiled in**, and that is why an IP change means a rebuild.

Application	Where the address lives	When your IP changes
Web API (IIS)	Nowhere. IIS listens on <i>all</i> network interfaces, so it serves on whatever address the machine happens to have.	Nothing to do.
Web app	<code>web/.env.production</code> — Vite bakes the value into the JavaScript bundle at build time.	Edit the file, then redploy .
Android app	<code>mobile/local.properties</code> — Gradle compiles it into <code>BuildConfig.API_BASE_URL</code> .	Edit the file, then rebuild the APK .

Redeploying the web app without editing the file achieves nothing

The old IP address is inside the compiled JavaScript. Rebuilding without changing `.env.production` simply bakes the same wrong address in again. The site will load and then fail on every API call, which looks like a broken application rather than a wrong address. **Edit first, deploy second.**

One thing you never have to touch: **CORS**. The API does not hold a fixed list of allowed origins — it tests each one with a rule that accepts loopback and any private address in the `192.168.x.x`, `10.x.x.x` and `172.16–31.x.x` ranges. Whatever address your router gives you is accepted automatically.

PART 4

Unzip the project and find your address

Where to put it

Unzip somewhere with a **short path and no spaces**. Long paths break Gradle on Windows, and spaces break some of the command lines below.

```
Good: C:\Dev\VoltShare
Bad:  C:\Users\You\OneDrive\My Documents\Uni Work\SE4040 Assignment 1\VoltShare
```

Do not keep it inside OneDrive or Google Drive

Cloud sync tools lock files while they upload. Gradle and MSBuild write thousands of files during a build and will fail with permission errors that look like something else entirely.

What is in the zip that is not in Git

These files are deliberately excluded from version control because they hold secrets or machine-specific values. You have them only because the zip was shared with you directly — you would not get them from a `git clone`.

File	What it holds
<code>backend/src/VoltShare.Api/appsettings.Production.json</code>	The MongoDB Atlas connection string, the JWT signing key and the QR signing key.
<code>mobile/local.properties</code>	The Android SDK path, the Google Maps API key, and the API address.

Never commit these two files

They are already listed in `.gitignore` and have never been committed. Keep it that way — a database password in Git history is very hard to remove, and the marker can see your repository.

Find your laptop's address

Open PowerShell and run:

```
ipconfig

Wireless LAN adapter Wi-Fi:
    IPv4 Address. . . . . : 192.168.1.190    <- this one
```

Write that number down. Every step from here uses it. Note it is the address under **Wi-Fi** — ignore any 169.254.x.x addresses, which mean a disconnected adapter.

Everyone and every device must be on the same router

Your laptop, your phone and anyone testing your site all have to be on the **same Wi-Fi network**. A phone on mobile data cannot reach 192.168.x.x — those addresses only exist inside your own network. If you are demonstrating to someone, put them on your Wi-Fi first.

Stop the address moving

Your router leases that address to you and can hand out a different one after a reboot. If that happens, the web app and the Android app both stop working until you rebuild them.

Worth two minutes before a demonstration: open your router's admin page (usually `http://192.168.1.1`), find **DHCP reservation**, **Static DHCP** or **Bind IP to MAC**, and pin your laptop's address. Your MAC address comes from `getmac /v`.

PART 5

Change three files for your machine

The zip is configured for the original machine at 192.168.1.190. Replace that with your own address in the three places below. Everywhere in this section, **192.168.1.190** means **your** address.

5.1 — The web application

File: `web/.env.production`

```
VITE_API_BASE_URL=http://192.168.1.190:8080/api/v1
```

Change the IP only. Keep the port 8080 and the path `/api/v1`. This file is tracked by Git, so if you commit your change, remember your teammates will need theirs back.

5.2 — The Android application

File: `mobile/local.properties`

```
API_BASE_URL=http://192.168.1.190:8080/api/v1/
```

Keep the trailing slash

Retrofit refuses a base address that does not end in `/` and throws `IllegalArgumentException: baseUrl must end in /`. Note the web app's value has **no** trailing slash and the mobile one **does**. That is correct, not a typo.

If you leave this line out entirely, the build falls back to `http://10.0.2.2:8080/api/v1/`, which is the alias the Android emulator uses for the machine hosting it. That works on an emulator and fails on a real

phone.

5.3 — The API's list of allowed origins

File: backend/src/VoltShare.Api/appsettings.Production.json

```
"Cors": {
  "AllowedOrigins": [
    "http://localhost:8081",
    "http://192.168.1.190:8081",
    "http://localhost:5173"
  ]
}
```

Add your own address to the list. Strictly speaking the API already accepts any private address through a rule in code, so this is belt and braces — but keeping the list accurate makes the configuration self-documenting, and it is the first place anyone will look if a browser reports a CORS error.

Summary of the three edits

File	Line to change	Trailing slash?
web/.env.production	VITE_API_BASE_URL	No
mobile/local.properties	API_BASE_URL	Yes
backend/.../appsettings.Production.json	Cors.AllowedOrigins	No

PART 6

Deploy the Web API

One script does everything: compiles the API, creates the IIS application pool and website, sets folder permissions, opens the firewall port and verifies the result. Run it again any time you change backend code — it refreshes the files rather than creating a duplicate site.

Both deploy scripts must run as Administrator

Creating IIS sites and setting folder permissions require it. Press **Start**, type **PowerShell**, right-click **Windows PowerShell** and choose **Run as administrator**. The title bar will say *Administrator*.

In that elevated window:

```
cd C:\Dev\VoltShare\scripts
powershell -ExecutionPolicy Bypass -File .\deploy-api-iis.ps1
```

-ExecutionPolicy Bypass is needed because Windows blocks unsigned scripts by default. It applies to this one run only and changes nothing permanently.

What you should see at the end

```
=====
VoltShare API deployed successfully
=====
API           : http://localhost:8080
On the network : http://192.168.1.190:8080
Health check   : http://localhost:8080/api/v1/health
=====
```

Check it

Open this in a browser:

```
http://localhost:8080/api/v1/health
```

You want to see:

```
{"status": "Healthy", "service": "VoltShare.Api",
  "database": "Connected", "serverTimeUtc": "..."}

```

"database": "Disconnected" or a 503

The API is running but cannot reach MongoDB Atlas. That is almost always the Atlas IP allowlist — see **Part 12**. It is not a problem with your deployment.

PART 7

Deploy the web application

Same pattern. This script runs `npm run build`, copies the compiled `dist` folder into IIS, creates the site on port 8081 and opens the firewall.

Edit `web/.env.production` first

Step 5.1. If you skip it, the build compiles the wrong IP address into the JavaScript and the site will fail on every API call while looking perfectly healthy.

First time only, install the npm packages:

```
cd C:\Dev\VoltShare\web
npm install
```

Then, from the **elevated** PowerShell window:

```
cd C:\Dev\VoltShare\scripts
powershell -ExecutionPolicy Bypass -File .\deploy-web-iis.ps1
```

What you should see

```
=====
VoltShare Web deployed successfully
=====
Web application : http://localhost:8081
On the network  : http://192.168.1.190:8081
Files           : C:\inetpub\VoltShareWeb

IIS runs as a Windows service, so both sites now start
automatically with the machine. Nothing to launch by hand.
=====
```

That last line matters: from now on you do **not** start anything manually. Both sites come up with the machine. `npm run dev` is now purely a coding tool and plays no part in how anyone reaches the application.

Sign in

Open `http://localhost:8081` and sign in with the back-office account:

Role	Email	Password
Back office	admin@voltshare.lk	Admin@123
Grid operator	operator1@voltshare.lk	Operator@123
Prosumer (mobile app only)	kamal@example.lk	Prosumer@123

A prosumer account signing in on the web is turned away on purpose — prosumers belong to the mobile application.

PART 8 · NEARLY EVERYONE HITS THIS

The firewall trap

Everything works on your own laptop. You type the network address into your phone and get:

```
This site can't be reached
192.168.1.190 unexpectedly closed the connection.
ERR_CONNECTION_CLOSED
```

Why

Windows Firewall keeps **three separate sets of rules** — Domain, Private and Public — and applies only the set matching the network you are currently on. The deploy scripts open ports 8080 and 8081 for **Private** networks.

Windows classifies a network as **Public** whenever you skip the “Do you want your PC to be discoverable?” prompt on first connecting. That is a sensible default for a café. It is wrong for your own router — and it means the firewall rules the scripts created are simply not in effect.

Check

```
Get-NetConnectionProfile | Select-Object Name, NetworkCategory
```

```
Name                NetworkCategory
----                -
SLT-4G_164F41      Public          <- this is the problem
```

Fix

In the **elevated** PowerShell window:

```
Set-NetConnectionProfile -InterfaceAlias "Wi-Fi" -NetworkCategory Private
```

Takes effect immediately. No restart, no redeploy. Reload the page on your phone.

What this changes, and how to undo it

It tells Windows to trust this network the way it trusts a home LAN, which is accurate for your own router. Fully reversible — swap Private for Public. **Do change it back** before joining a public café or campus network.

Verify

```
Test-NetConnection -ComputerName 192.168.1.190 -Port 8081

TcpTestSucceeded : True          <- the firewall is now letting it through
```

If it still fails after that

Then Windows was not the only obstacle, and the suspect is your router. Many home and 4G routers ship with **AP isolation** (also called client isolation or guest mode) switched on, which stops devices on the same Wi-Fi from talking to each other at all, whatever your laptop permits. Look for it under the wireless settings in the router admin page.

How to tell the two apart: if Test -NetConnection succeeds from the laptop but the phone still cannot connect, it is the router, not Windows.

PART 9

Build and run the Android app

Open the right folder

In Android Studio choose **Open** and select the **mobile** folder — not the repository root. Android Studio needs to see `settings.gradle.kts` at the top level.

```
C:\Dev\VoltShare\mobile    <- open this
C:\Dev\VoltShare           <- not this
```

The first Gradle sync downloads a lot and can take several minutes. Let it finish.

Set your API address

Step 5.2 — `mobile/local.properties`. After editing it, click **Sync Now** on the banner Android Studio shows, because Gradle has to regenerate `BuildConfig`.

Run it

Target	What to do	Which address
Emulator	Device Manager → Create device . Pick a Pixel with a Play Store image so Google Maps works. Press Run .	Either your LAN IP or <code>10.0.2.2</code> . Both reach your PC.
Real phone	Enable Developer options (tap Build number seven times) and USB debugging . Plug in, press Run .	Must be your LAN IP. The phone must be on the same Wi-Fi.

Build an APK to share

```
cd C:\Dev\VoltShare\mobile
.\gradlew.bat assembleDebug
```

The file appears at `app\build\outputs\apk\debug\app-debug.apk`. Install it on a plugged-in phone with:

```
adb install -r app\build\outputs\apk\debug\app-debug.apk
```

Or copy the file to the phone and tap it, allowing installation from unknown sources when prompted.

Why 10.0.2.2 exists at all

The emulator runs behind a virtual router. Inside it, `localhost` means the *emulator*, not your PC, so nothing is listening. `10.0.2.2` is a special alias the emulator maps to the host machine. A real phone has no idea what that address means, which is why it needs your actual LAN address.

PART 10

Check that everything works

Work down the list. Each row depends on the ones above it, so fix them in order rather than skipping ahead.

#	Test	Expected
1	http://localhost	The blue IIS default page. Proves IIS is running.
2	http://localhost:8080/api/v1/health	"status": "Healthy" and "database": "Connected".
3	http://localhost:8080/swagger	The Swagger UI listing every endpoint.
4	http://localhost:8081	The VoltShare sign-in page. Sign in as admin.
5	http://YOUR-IP:8081 on the laptop	The same page. Proves IIS answers on the network address.
6	http://YOUR-IP:8081 on your phone	The same page. If this fails, go to Part 8 .
7	Sign in on the phone's browser	Reaches the dashboard. Proves the browser can reach the API on port 8080 too.
8	Android app → sign in as kamal@example.lk	The prosumer home screen with live counts.
9	Android app → Find nodes	Two microgrid stations listed.
10	Android app → Book energy transfer	Windows listed, and a booking can be created.

Step 6 is the real test

Steps 1 to 5 all pass even when the firewall is blocking everything, because traffic from your own machine never crosses it. The first genuine proof that you are hosting anything is a **different device** loading the page. Do that before you record your demonstration video.

PART 11

Troubleshooting

Symptom	Cause	Fix
HTTP 500.19 from the API	The ASP.NET Core Hosting Bundle is missing, or was installed before IIS was switched on.	Install or repair it, then run <code>iisreset</code> .
ERR_CONNECTION_CLOSED or a timeout from another device	Firewall rules are scoped to Private; your network is classified Public.	Part 8. Set the network to Private.
Health check says database: Disconnected	Your public IP is not on the MongoDB Atlas allowlist.	Part 12. Only the Atlas account owner can add it.
Web app loads but every action fails; console shows Failed to fetch	The wrong API address was compiled into the bundle.	Fix <code>.env.production</code> , then redeploy. Editing alone does nothing.
CORS error in the browser console	The site is being opened by a name the API does not recognise as private.	Use the plain <code>192.168.x.x</code> address, not a hostname.
Android: CLEARTEXT communication not permitted	A release build pointed at an address that is not in the whitelist.	Use the debug build, which permits cleartext to any host.
Android: baseUrl must end in /	<code>API_BASE_URL</code> has no trailing slash.	Add it. Step 5.2.
Android app cannot reach the service	Phone on mobile data, or on a different Wi-Fi.	Same router as the laptop. Turn mobile data off to be sure.
Map screen is blank	Maps SDK not enabled, or your machine's SHA-1 is not on the key.	Part 12.
It all worked yesterday, nothing works today	The router handed your laptop a different IP address.	<code>ipconfig</code> , then redo Part 5 and redeploy.
Gradle fails with odd permission errors	The project is inside OneDrive or another sync folder.	Move it to <code>C:\Dev\</code> .

PART 12

Two things only the repository owner can do

These are account-level settings on services the team shares. Nobody can fix them from their own laptop — send them to whoever owns the MongoDB Atlas and Google Cloud accounts.

12.1 — MongoDB Atlas network access

Atlas refuses connections from any address that is not on its allowlist. Each teammate sits behind a different home router with a different **public** IP address, so a database that works for one person will time out for everyone else.

What the owner does

- 1 Sign in at `cloud.mongodb.com`.
- 2 Open the cluster and choose **Network Access** in the left sidebar.

- 3 **Add IP Address**, and either add each teammate's public address (they can read it from `whatismyip.com`), or click **Allow access from anywhere**, which adds `0.0.0.0/0`.

0.0.0.0/0 is acceptable here, briefly

It means any address on the internet may attempt to connect — they still need the username and password. For a student project on a free tier that is a reasonable trade against four people editing an allowlist every time their router reboots. **Remove it and rotate the password when the module ends.**

Symptom when this is wrong: the health check returns `"database": "Disconnected"`, or the API returns 503. The API itself is fine.

12.2 — The Google Maps API key

The map screen needs two things that are not in the repository.

Enable the SDK and billing

In the Google Cloud console: enable **Maps SDK for Android** for the project, and attach a billing account. Google requires a billing account even inside the free tier, and without it the map returns `INVALID_ARGUMENT` and renders as a blank grid.

Add each teammate's signing fingerprint

An Android API key is normally restricted to a package name plus the SHA-1 fingerprint of the keystore that signed the app. Every developer has a **different** debug keystore, so a key restricted to one person's fingerprint shows a blank map for everyone else.

Each teammate runs this and sends their SHA-1 to the owner:

```
cd C:\Dev\VoltShare\mobile
.\gradlew.bat signingReport
```

Look for the debug variant and copy the SHA1 line. The owner adds each fingerprint under the key's Android restrictions, alongside the package name `lk.sliit.voltshare`.

The quicker alternative, for a shared team key: leave the key unrestricted while developing and restrict it before submission. Less safe, but the key is not doing anything valuable.

Before submission

- Replace `<IT Number - Member Name>` in every source file. The brief says files without a header comment block are not marked.
- Change the seeded `Admin@123` password.
- Rotate the MongoDB Atlas password and remove `0.0.0.0/0` from the allowlist.
- Restrict the Google Maps key to the package name and the fingerprints in use.
- Confirm `appsettings.Production.json` and `local.properties` are still untracked.

If something here is wrong or out of date, fix it and tell the rest of the team. A setup guide that lies is worse than no guide at all.